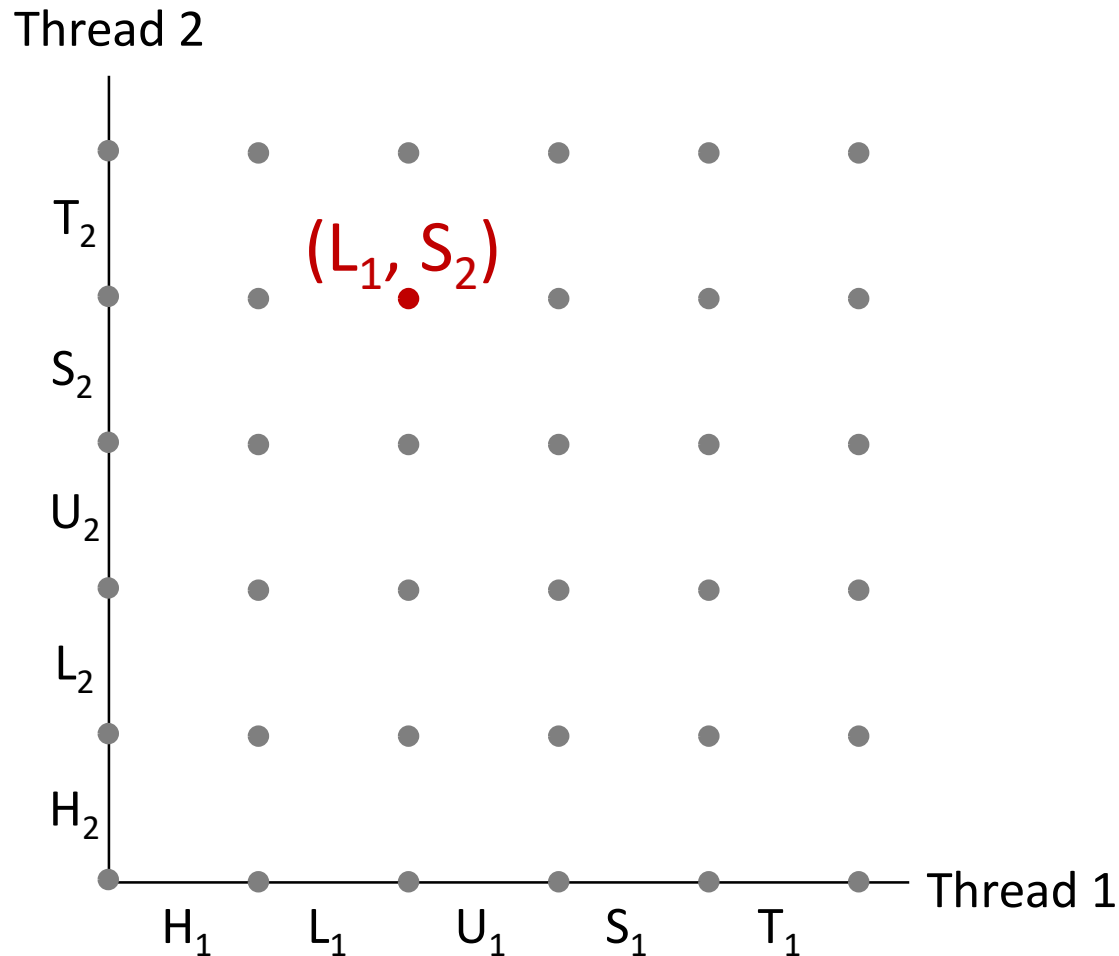


Progress Graphs



A *progress graph* depicts the discrete *execution state space* of concurrent threads.

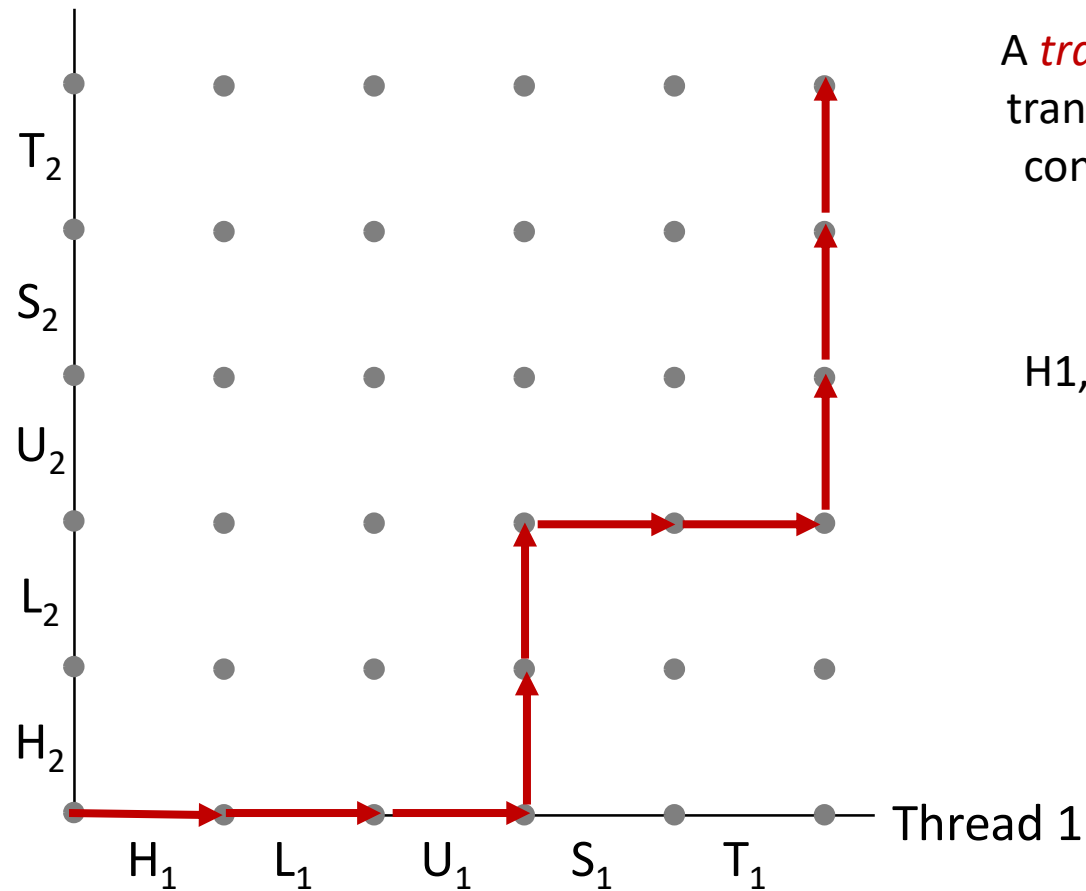
Each axis corresponds to the sequential order of instructions in a thread.

Each point corresponds to a possible *execution state* $(Inst_1, Inst_2)$.

E.g., (L_1, S_2) denotes state where thread 1 has completed L_1 and thread 2 has completed S_2 .

Trajectories in Progress Graphs

Thread 2

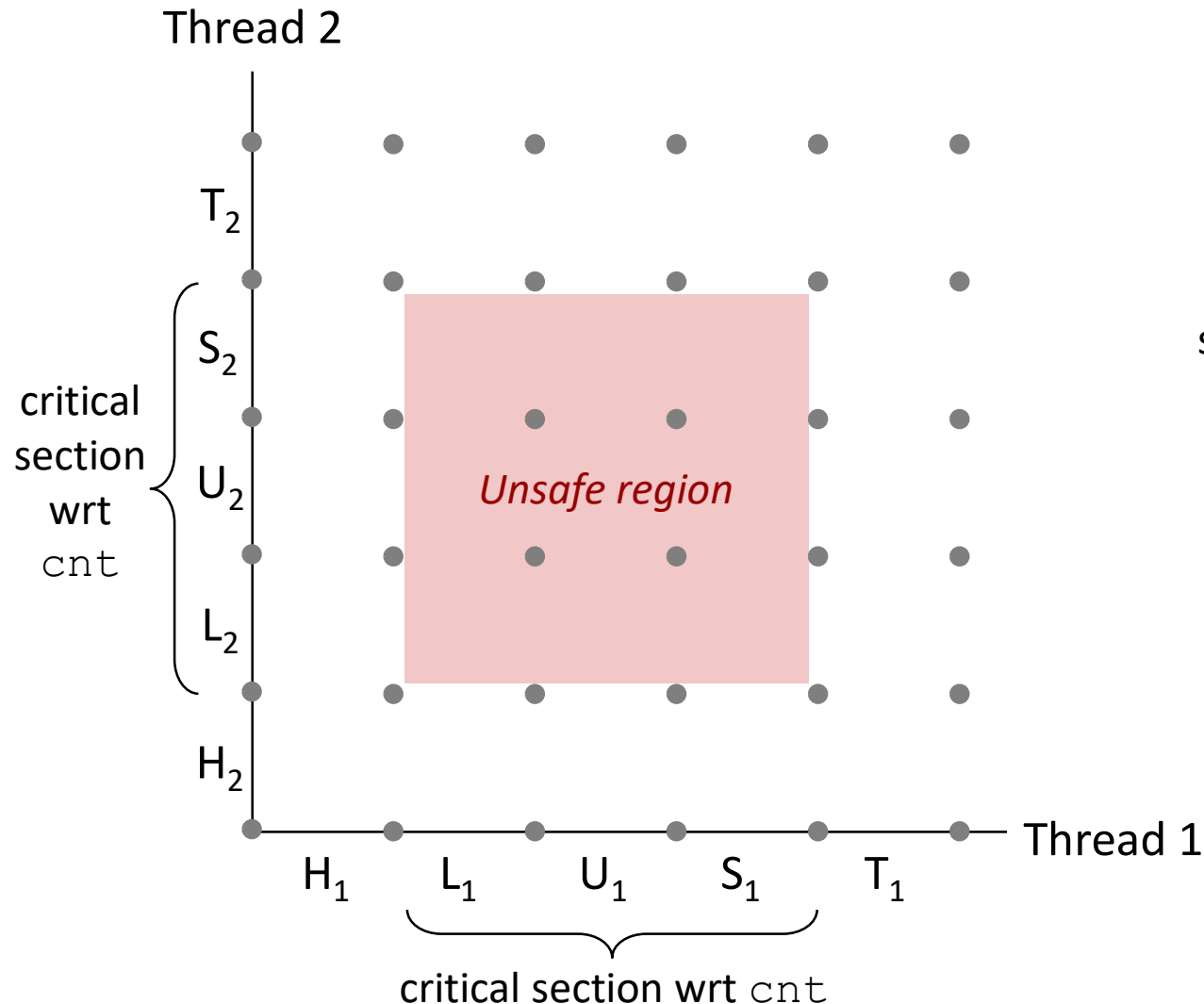


A *trajectory* is a sequence of legal state transitions that describes one possible concurrent execution of the threads.

Example:

$H_1, L_1, U_1, H_2, L_2, S_1, T_1, U_2, S_2, T_2$

Critical Sections and Unsafe Regions

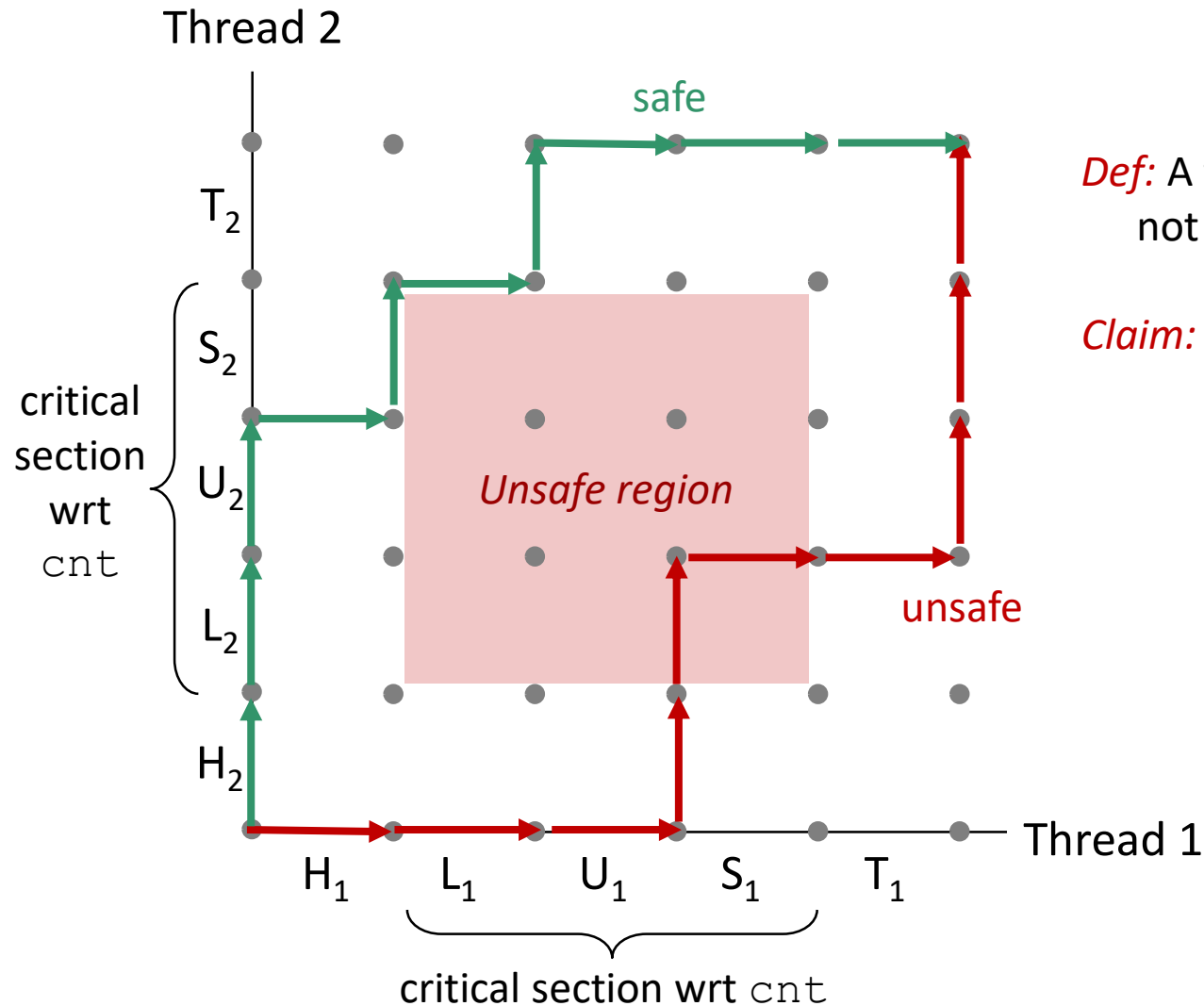


L , U , and S form a *critical section* with respect to the shared variable `cnt`

Instructions in critical sections (wrt to some shared variable) should not be interleaved

Sets of states where such interleaving occurs form *unsafe regions*

Critical Sections and Unsafe Regions



Def: A trajectory is *safe* iff it does not enter any unsafe region

Claim: A trajectory is correct (wrt cnt) iff it is safe

Enforcing Mutual Exclusion

- **Question:** How can we guarantee a safe trajectory?
- **Answer:** We must *synchronize* the execution of the threads so that they never have an unsafe trajectory.
 - i.e., need to guarantee *mutually exclusive access* to critical regions
- **Classic solution:**
 - Semaphores (Edsger Dijkstra)
- **Other approaches (out of our scope)**
 - Mutex and condition variables (Pthreads)
 - Monitors (Java)

Semaphores

- ***Semaphore***: non-negative global integer synchronization variable
- **Manipulated by P and V operations:**
 - $P(s)$: [**while** ($s == 0$) **wait()**; $s--$;]
 - Dutch for "Proberen" (test)
 - $V(s)$: [$s++$;]
 - Dutch for "Verhogen" (increment)
- **OS kernel guarantees that operations between brackets [] are executed indivisibly**
 - Only one P or V operation at a time can modify s .
 - When **while** loop in P terminates, only that P can decrement s
- ***Semaphore invariant***: ($s \geq 0$)

Using Semaphores for Mutual Exclusion

■ Basic idea:

- Associate a unique semaphore *mutex*, initially 1, with each shared variable (or related set of shared variables).
- Surround corresponding critical sections with $P(mutex)$ and $V(mutex)$ operations.

■ Terminology:

- *Binary semaphore*: semaphore whose value is always 0 or 1
- *Mutex*: binary semaphore used for mutual exclusion
 - P operation: “locking” the mutex
 - V operation: “unlocking” or “releasing” the mutex
 - “Holding” a mutex: locked and not yet unlocked.
- *Counting semaphore*: used as a counter for set of available resources.

Exploiting parallel execution

- **So far, we've used threads to deal with I/O delays**
 - e.g., one thread per client to prevent one from delaying another
- **Multi-core CPUs offer another opportunity**
 - Spread work over threads executing in parallel on N cores
 - Happens automatically, if many independent tasks
 - e.g., running many applications or serving many clients
 - Can also write code to make one big task go faster
 - by organizing it as multiple parallel sub-tasks
- **Shark machines can execute 16 threads at once**
 - 8 cores, each with 2-way hyperthreading (not covered)
 - Theoretical speedup of 16X
 - never achieved in our benchmarks

Summation Example

- **Sum numbers 0, ..., N-1**
 - Should add up to $(N-1)*N/2$
- **Partition into K ranges**
 - $\lfloor N/K \rfloor$ values each
 - Accumulate leftover values serially
- **Method #1: All threads update single global variable**
 - 1A: No synchronization
 - 1B: Synchronize with pthread semaphore
 - 1C: Synchronize with pthread mutex
 - “Binary” semaphore. Only values 0 & 1

Amdahl's Law

■ Overall problem

- T Total sequential time required
- p Fraction of total that can be sped up ($0 \leq p \leq 1$)
- k Speedup factor

■ Resulting Performance

- $T_k = pT/k + (1-p)T$
 - Portion which can be sped up runs k times faster
 - Portion which cannot be sped up stays the same
- Maximum possible speedup
 - $k = \infty$
 - $T_\infty = (1-p)T$

Amdahl's Law Example

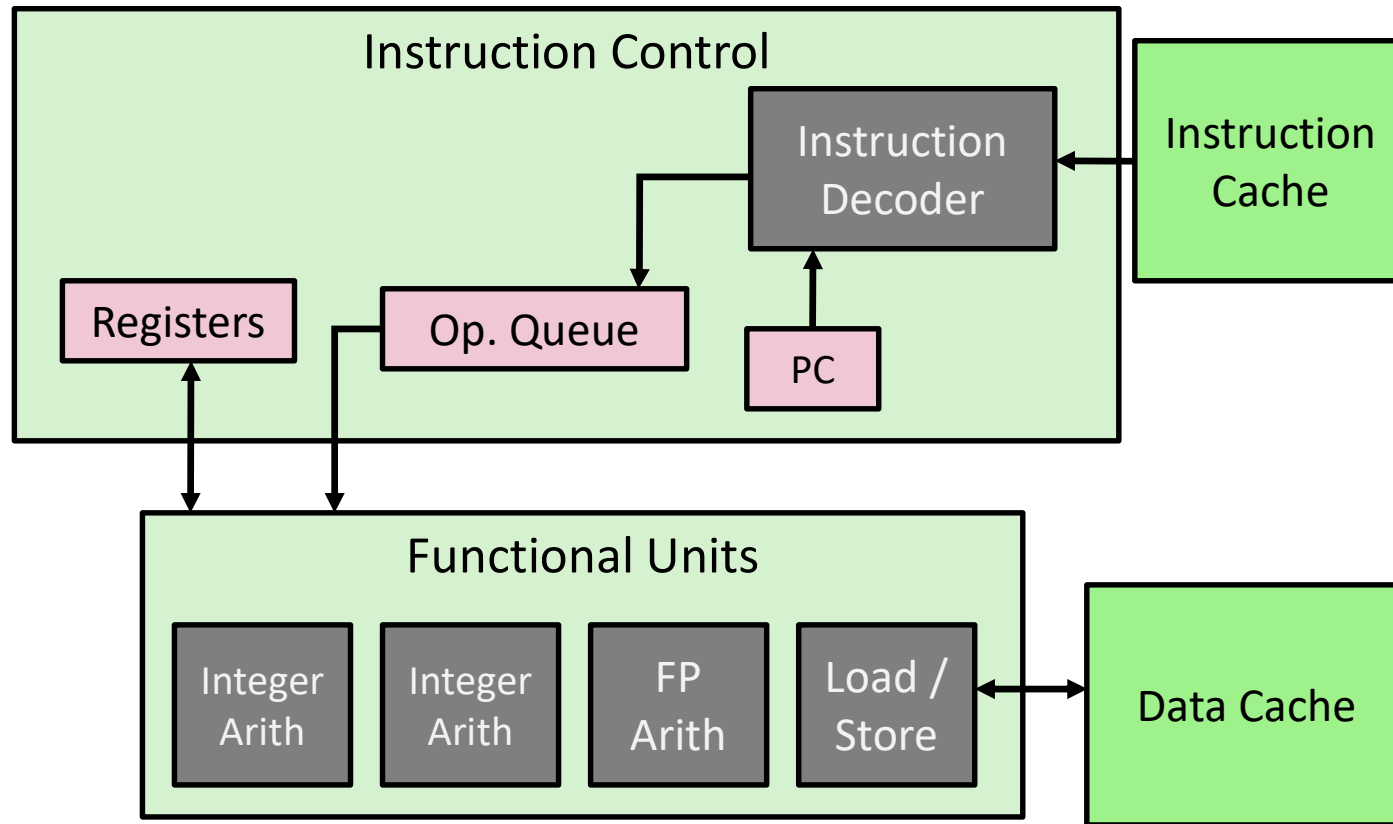
■ Overall problem

- $T = 10$ Total time required
- $p = 0.9$ Fraction of total which can be sped up
- $k = 9$ Speedup factor

■ Resulting Performance

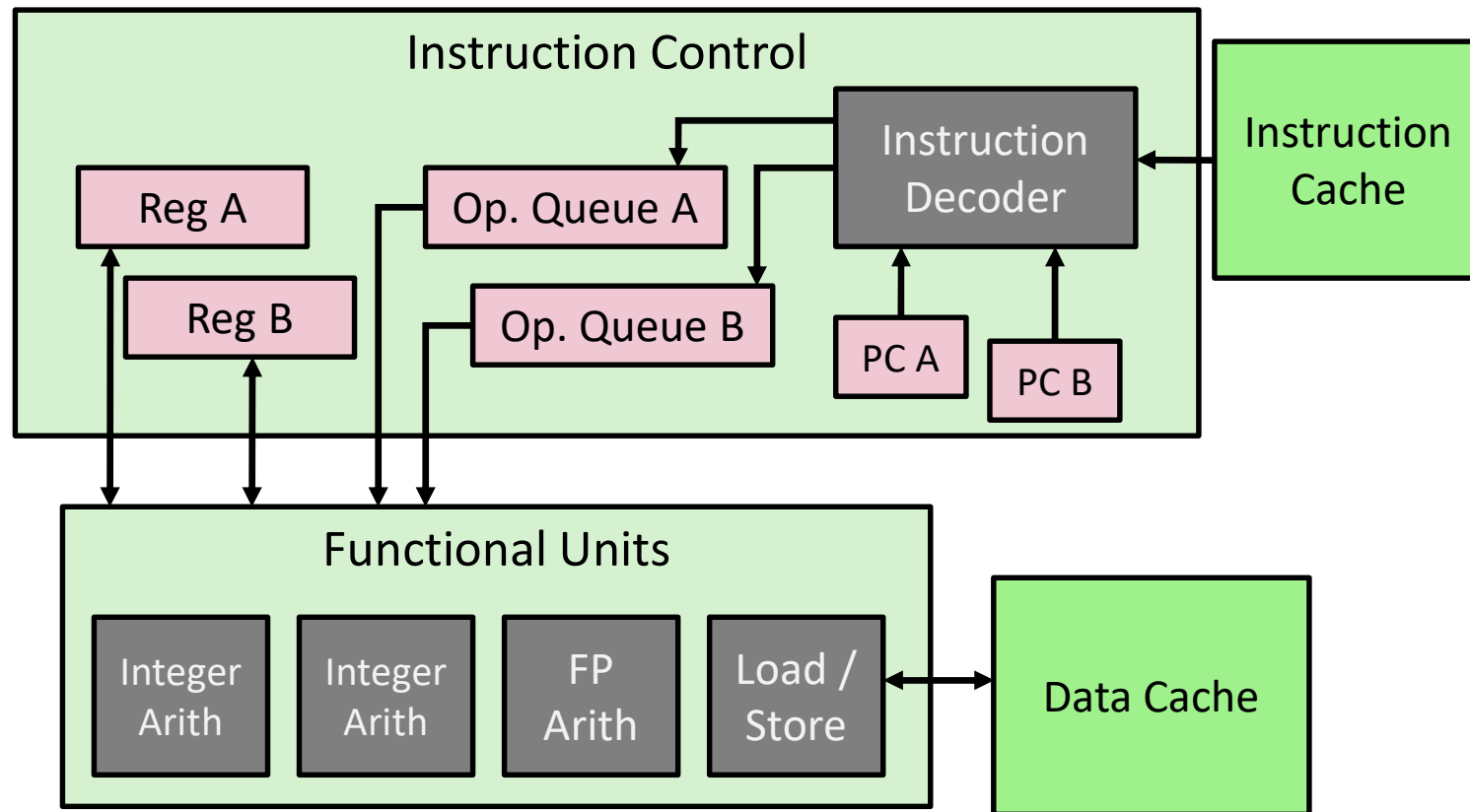
- $T_9 = 0.9 * 10/9 + 0.1 * 10 = 1.0 + 1.0 = 2.0$
- Maximum possible speedup
 - $T_{\infty} = 0.1 * 10.0 = 1.0$

Out-of-Order Processor Structure



- Instruction control dynamically converts program into stream of operations
- Operations mapped onto functional units to execute in parallel

Hyperthreading



- Replicate enough instruction control to process K instruction streams
- K copies of all registers
- Share functional units

Flynn's Taxonomy of Computers

- Mike Flynn, “**Very High-Speed Computing Systems**,” Proc. of IEEE, 1966
- **SISD**: Single instruction operates on single data element
- **SIMD**: Single instruction operates on multiple data elements
 - Array processor
 - Vector processor
- **MISD**: Multiple instructions operate on single data element
 - Closest form: systolic array processor, streaming processor
- **MIMD**: Multiple instructions operate on multiple data elements (multiple instruction streams)
 - Multiprocessor
 - Multithreaded processor

Data Parallelism

- Concurrency arises from performing the **same operation on different pieces of data**
 - Single instruction multiple data (SIMD)
 - E.g., dot product of two vectors
- **Contrast with data flow**
 - Concurrency arises from executing different operations in parallel (in a data driven manner)
- **Contrast with thread (“control”) parallelism**
 - Concurrency arises from executing different threads of control in parallel
- **SIMD exploits operation-level parallelism on different data**
 - Same operation concurrently applied to different pieces of data
 - A form of ILP where instruction happens to be the same across data

SIMD Processing Paradigm

- **Single instruction operates on multiple data elements**
 - In time or in space
- **Multiple processing elements (PEs), i.e., execution units**
- **Time-space duality**
 - **Array processor**: Instruction operates on multiple data elements at the same time using different spaces (PEs)
 - **Vector processor**: Instruction operates on multiple data elements in consecutive time steps using the same space (PE)

Vector/SIMD Processing Summary

- Vector/SIMD machines are good at exploiting **regular data-level parallelism**
 - Same operation performed on many data elements
 - Improve performance, simplify design (no intra-vector dependencies)
- **Performance improvement limited by vectorizability of code**
 - Scalar operations limit vector machine performance
 - Remember **Amdahl's Law**
 - CRAY-1 was the fastest SCALAR machine at its time!
- **Many existing ISAs include (vector-like) SIMD operations**
 - Intel MMX/SSEn/AVX, PowerPC AltiVec, ARM Advanced SIMD